

Correction, réflexions et extensions

Exercices d'optimisation avec Python

Table des matières

1	Optimisation sans contraintes	2
1.1	Exercice 1	2
1.2	Exercice 2	3
2	Optimisation avec contraintes	4
2.1	Exercice 3	4
2.2	Exercice 4	5
3	Programmation linéaire	7
3.1	Exercice 5	7
3.2	Exercice 6	8

1 Optimisation sans contraintes

1.1 Exercice 1

On souhaite minimiser la fonction :

$$f(x, y) = (x - 1)^2 + (y - 2)^2 + xy$$

Correction

La fonction peut être implémentée ainsi :

```
from scipy.optimize import minimize

def f(v):
    x, y = v
    return (x - 1)**2 + (y - 2)**2 + x*y

x0 = [0, 0]

result = minimize(f, x0, method="BFGS")

print("Succes :", result.success)
print("Message :", result.message)
print("Solution optimale :", result.x)
print("Valeur minimale :", result.fun)
```

La solution optimale attendue est :

$$x = 0, \quad y = 2$$

La valeur minimale est :

$$f(0, 2) = (0 - 1)^2 + (2 - 2)^2 + 0 \times 2 = 1$$

On peut vérifier analytiquement en calculant les dérivées partielles :

$$\frac{\partial f}{\partial x} = 2(x - 1) + y = 2x + y - 2$$

$$\frac{\partial f}{\partial y} = 2(y - 2) + x = x + 2y - 4$$

À l'optimum :

$$2x + y - 2 = 0$$

$$x + 2y - 4 = 0$$

Donc :

$$2x + y = 2$$

$$x + 2y = 4$$

La résolution du système donne :

$$x = 0, \quad y = 2$$

Réflexion

Sans le terme xy , la fonction aurait naturellement son minimum en $(1, 2)$, car les termes quadratiques sont centrés sur ce point.

Question :

Pourquoi le minimum n'est-il pas exactement au point $(1, 2)$?

Extensions

- Tracer les courbes de niveau de la fonction.
- Comparer les méthodes BFGS, Nelder-Mead et CG.
- Fournir explicitement le gradient à minimizer.
- Tester plusieurs points initiaux.
- Supprimer le terme xy , puis comparer la nouvelle solution avec la solution initiale.

1.2 Exercice 2

On souhaite minimiser la fonction de coût :

$$C(x, y) = x^2 + y^2 - 3x - 4y + xy$$

Correction

```
from scipy.optimize import minimize

def C(v):
    x, y = v
    return x**2 + y**2 - 3*x - 4*y + x*y

x0 = [0, 0]

result = minimize(C, x0, method="BFGS")

print("Succes :", result.success)
print("Message :", result.message)
print("Solution optimale :", result.x)
print("Valeur minimale :", result.fun)
```

La solution optimale attendue est :

$$x = \frac{2}{3}, \quad y = \frac{5}{3}$$

Soit environ :

$$x \approx 0,6667, \quad y \approx 1,6667$$

La valeur minimale est :

$$C\left(\frac{2}{3}, \frac{5}{3}\right) = \left(\frac{2}{3}\right)^2 + \left(\frac{5}{3}\right)^2 - 3\left(\frac{2}{3}\right) - 4\left(\frac{5}{3}\right) + \frac{2}{3} \times \frac{5}{3}$$

$$C\left(\frac{2}{3}, \frac{5}{3}\right) = -\frac{13}{3} \approx -4,3333$$

Vérification analytique :

$$\frac{\partial C}{\partial x} = 2x + y - 3$$

$$\frac{\partial C}{\partial y} = x + 2y - 4$$

À l'optimum :

$$2x + y = 3$$

$$x + 2y = 4$$

Ce système donne :

$$x = \frac{2}{3}, \quad y = \frac{5}{3}$$

Réflexion

Dans le contexte d'analyse de logs, x et y peuvent représenter deux flux ou deux paramètres à ajuster. La solution optimale donne la configuration qui minimise le coût global.

Question :

Si x et y représentent des quantités indivisibles, par exemple un nombre de serveurs ou un nombre de tâches, la solution décimale est-elle directement utilisable ?

Cette question permet d'introduire la différence entre optimisation continue et optimisation entière.

Extensions

- Ajouter les contraintes $x \geq 0$ et $y \geq 0$.
- Imposer que x et y soient des entiers.
- Comparer la solution continue avec une solution arrondie.
- Modifier le coefficient du terme xy et observer l'effet sur la solution.

2 Optimisation avec contraintes

2.1 Exercice 3

On souhaite minimiser :

$$f(x, y) = (x - 2)^2 + (y + 1)^2$$

sous la contrainte :

$$x + 2y = 3$$

Correction

```
from scipy.optimize import minimize

def f(v):
    x, y = v
    return (x - 2)**2 + (y + 1)**2

constraints = {
    "type": "eq",
    "fun": lambda v: v[0] + 2*v[1] - 3
}

x0 = [0, 0]

result = minimize(f, x0, method="SLSQP", constraints=constraints)

print("Succes :", result.success)
print("Message :", result.message)
print("Solution optimale :", result.x)
print("Valeur minimale :", result.fun)
print("Verification contrainte :", result.x[0] + 2*result.x[1])
```

La solution optimale attendue est :

$$x = \frac{13}{5} = 2,6$$

$$y = \frac{1}{5} = 0,2$$

La valeur minimale est :

$$f(2,6, 0,2) = (2,6 - 2)^2 + (0,2 + 1)^2$$

$$f(2,6, 0,2) = 0,6^2 + 1,2^2 = 0,36 + 1,44 = 1,8$$

Vérification de la contrainte :

$$x + 2y = 2,6 + 2(0,2) = 3$$

La contrainte est donc respectée.

Réflexion

Sans contrainte, la fonction est minimale au point :

$$(2, -1)$$

Mais ce point ne respecte pas la contrainte :

$$2 + 2(-1) = 0 \neq 3$$

L'optimisation cherche donc le point le plus proche de $(2, -1)$ qui appartient à la droite :

$$x + 2y = 3$$

Question :

Comment la contrainte modifie-t-elle la solution optimale ?

L'objectif est de faire comprendre que la contrainte réduit l'espace des solutions possibles.

Extensions

- Tracer la fonction et la droite de contrainte.
- Comparer la solution avec et sans contrainte.
- Ajouter une contrainte supplémentaire : $x \geq 0$.
- Tester plusieurs points initiaux et observer la stabilité du résultat.

2.2 Exercice 4

On souhaite minimiser :

$$L(x, y) = 3x^2 + 2y^2 + xy$$

sous la contrainte :

$$x + y = 100$$

Correction

```
from scipy.optimize import minimize

def L(v):
    x, y = v
    return 3*x**2 + 2*y**2 + x*y

constraints = {
    "type": "eq",
    "fun": lambda v: v[0] + v[1] - 100
}
```

```

x0 = [50, 50]

result = minimize(L, x0, method="SLSQP", constraints=constraints)

print("Succes :", result.success)
print("Message :", result.message)
print("Solution optimale :", result.x)
print("Valeur minimale :", result.fun)
print("Verification contrainte :", result.x[0] + result.x[1])

```

La solution optimale attendue est :

$$x = 37,5, \quad y = 62,5$$

La valeur minimale est :

$$L(37,5, 62,5) = 3(37,5)^2 + 2(62,5)^2 + 37,5 \times 62,5$$

$$L(37,5, 62,5) = 14375$$

Vérification de la contrainte :

$$37,5 + 62,5 = 100$$

Réflexion

Une répartition équilibrée 50/50 n'est pas nécessairement optimale.

Ici, la fonction de coût pénalise davantage la variable x , car son coefficient quadratique est plus élevé :

$$3x^2$$

Cela explique pourquoi la solution affecte moins de charge à x et davantage à y .

Question :

Pourquoi la solution optimale n'est-elle pas $x = 50, y = 50$?

Cette question permet de distinguer une répartition intuitive d'une répartition mathématiquement optimale.

Extensions

- Ajouter les bornes $0 \leq x \leq 100$ et $0 \leq y \leq 100$.
- Ajouter un troisième serveur.
- Imposer que les charges soient entières.
- Comparer la solution continue avec une solution entière.
- Modifier les coefficients de la fonction de coût et observer le changement de répartition.

3 Programmation linéaire

3.1 Exercice 5

On souhaite résoudre :

$$\text{Maximiser } Z = 3x + 2y$$

sous les contraintes :

$$x + y \leq 4$$

$$x \leq 2$$

$$y \leq 3$$

$$x \geq 0, \quad y \geq 0$$

Correction

Comme `linprog` résout un problème de minimisation, on transforme :

$$\max Z = 3x + 2y$$

en :

$$\min -Z = -3x - 2y$$

```
from scipy.optimize import linprog
c = [-3, -2]
A_ub = [
    [1, 1],
    [1, 0],
    [0, 1]
]
b_ub = [4, 2, 3]
bounds = [(0, None), (0, None)]
result = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")
print("Succes :", result.success)
print("Message :", result.message)
print("Solution optimale :", result.x)
print("Valeur maximale :", -result.fun)
```

La solution optimale est :

$$x = 2, \quad y = 2$$

La valeur maximale est :

$$Z = 3(2) + 2(2) = 10$$

Attention :

$$\text{result.fun} = -10$$

Il faut donc afficher :

$$-\text{result.fun} = 10$$

Réflexion

En programmation linéaire, l'optimum se trouve souvent sur un sommet du domaine réalisable. Ici, la solution optimale se trouve à l'intersection de :

$$x = 2$$

et :

$$x + y = 4$$

Ce qui donne :

$$x = 2, \quad y = 2$$

Question :

Pourquoi la solution optimale d'un problème linéaire se trouve-t-elle souvent sur un sommet de la région réalisable ?

Cette question permet d'introduire le principe géométrique de la programmation linéaire.

Extensions

- Résoudre graphiquement le problème.
- Ajouter une nouvelle contrainte, par exemple $2x + y \leq 5$.
- Modifier les coefficients de la fonction objectif.
- Comparer la résolution graphique et la résolution avec `linprog`.
- Introduire les variables d'écart pour transformer le problème en forme standard.

3.2 Exercice 6

Un data center souhaite maximiser le gain obtenu par deux types de tâches.
On définit :

x = nombre de tâches CPU

y = nombre de tâches I/O

Fonction objectif :

Maximiser $Z = 5x + 4y$

Contraintes :

$$3x + 2y \leq 18$$

$$x + 2y \leq 12$$

$$x \geq 0, \quad y \geq 0$$

Correction

Comme `linprog` minimise, on écrit :

$$\min -Z = -5x - 4y$$

```
from scipy.optimize import linprog

c = [-5, -4]

A_ub = [
    [3, 2],
    [1, 2]
]

b_ub = [18, 12]

bounds = [(0, None), (0, None)]

result = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")

print("Succes :", result.success)
print("Message :", result.message)
print("Solution optimale :", result.x)
print("Valeur maximale :", -result.fun)
```

La solution continue attendue est :

$$x = 3, \quad y = 4,5$$

La valeur maximale est :

$$Z = 5(3) + 4(4,5)$$

$$Z = 15 + 18 = 33$$

Cette solution est correcte si les tâches peuvent être fractionnées.

Si les tâches doivent être entières, alors la solution $(3, 4, 5)$ n'est pas directement applicable.

Une bonne solution entière est :

$$x = 4, \quad y = 3$$

La valeur obtenue est :

$$Z = 5(4) + 4(3) = 20 + 12 = 32$$

Vérification des contraintes :

$$3(4) + 2(3) = 12 + 6 = 18$$

$$4 + 2(3) = 10 \leq 12$$

La solution entière $(4, 3)$ est donc réalisable.

Réflexion

La solution obtenue par `linprog` est une solution continue. Elle peut contenir des valeurs décimales.

Dans un data center, il faut se demander si une demi-tâche est réellement possible. Si les tâches sont indivisibles, alors le problème doit être formulé comme un problème d'optimisation entière.

Question :

Peut-on réellement exécuter 4,5 tâches I/O ? Si non, quel type d'optimisation faut-il utiliser ?

Cette question permet d'introduire la programmation linéaire entière.

Extensions

- Imposer que x et y soient entiers.
- Résoudre le problème par énumération des solutions entières.
- Ajouter une troisième ressource, par exemple la mémoire RAM.
- Ajouter un troisième type de tâche.
- Comparer la solution continue et la meilleure solution entière.
- Étudier l'effet d'une augmentation ou d'une diminution des capacités CPU et I/O.